vex-exp-pick-and-sort

A guide to the VEX EXP Clawbot pick-and-sort program

UNTESTED ON HARDWARE

Contents: README, bring-up checklist, assumptions, settings reference, references

Built from README.md and the files in docs/ by tools/build_guide.py.

Not affiliated with or endorsed by VEX Robotics.

vex-exp-pick-and-sort

A VEX EXP Clawbot that sorts balls by colour. It drives a path on a 300 mm grid board, finds a ball with a Distance Sensor, picks it up with the claw, reads its colour with an Optical Sensor, drops it in the box for that colour, and goes back to the path to look for the next one.

UNTESTED ON HARDWARE. Everything here has only run in the laptop simulator and the unit tests. Before running it on a robot, work through the bring-up checklist.

What it does

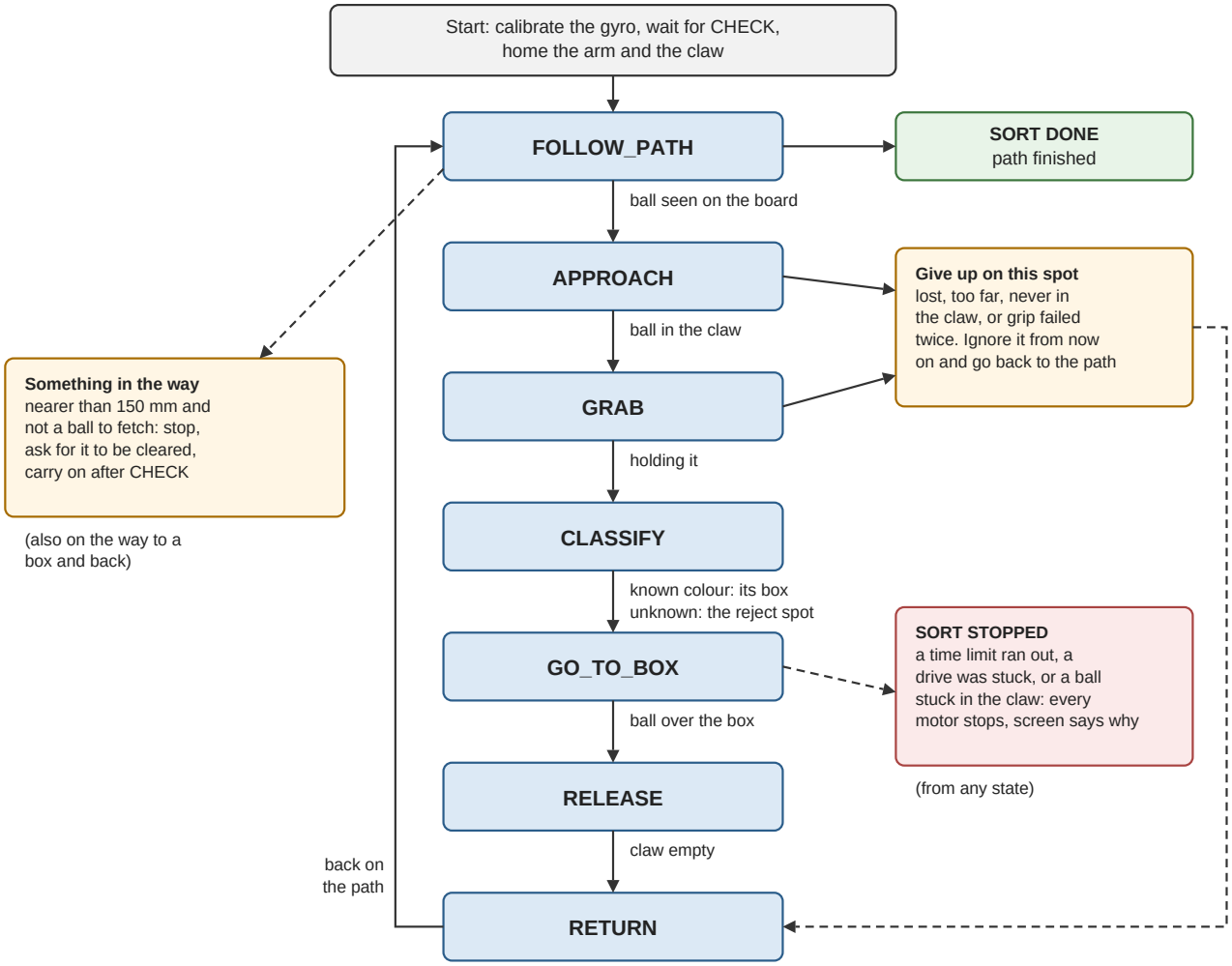
Milestone	What	State
M1	Project layout, settings file, hardware interface, path following, simulator, tests	Done, simulator only
M2	Distance and colour sensing, colour calibration	Done, simulator only
M3	Pick-and-sort behaviour with timeouts and recovery	Done, simulator only
M4	This guide, the wiring diagram and the PDF	Done

The program has five modes, chosen with `RUN_MODE` in `config/settings.py`: three bring-up checks, a plain drive along a path, and the sort itself. They are described under Running it.

The colour sorting is a calibrated rule, not machine learning. You record the Optical Sensor with the claw empty and with each ball, and the robot then picks the nearest recorded hue, or says "unknown" when nothing is close or the readings disagree. Nothing in the program learns or changes by itself while it runs.

How it works

The sort, state by state (RUN_MODE "sort")



Left or Right button on the Brain: everything stops, from any state.

The states of the sort and what happens when something goes wrong

The sort is a state machine in src/mission.py. The Brain screen shows the state and the colour the whole time.

State	What the robot does
FOLLOW_PATH	Drives `SORT_PATH`, watching the Distance Sensor
APPROACH	Drives slowly up to the ball, then creeps until the Optical Sensor in the claw sees it
GRAB	Closes the claw and checks it is holding something
CLASSIFY	Reads the colour
GO_TO_BOX	Lines up with that colour's box, raises the arm and moves in
RELEASE	Opens the claw and checks the ball has gone
RETURN	Backs out, lowers the arm and goes back to where it left the path

Every state has a time limit. When something goes wrong:

- **Object not found**: if it is lost from view, too far, or never reaches the claw, the robot gives up on that spot, goes back to the path and ignores anything near the spot from then on.
- **Grip failed**: it opens, backs off and tries once more, then gives up as above.
- **Colour unknown**: the ball goes to the reject spot, or the robot stops with a message if `UNKNOWN_COLOUR_ACTION` is `"stop"`.
- **Something in the way**: the robot stops, asks for the way to be cleared, and carries on after CHECK.
- **Anything else** (a stuck drive, a ball stuck in the claw, a time limit): every motor stops and the screen says why.

Only objects on the board count as balls. The boxes sit off the board, so when the Distance Sensor sees one (it does, from column D) the robot ignores it.

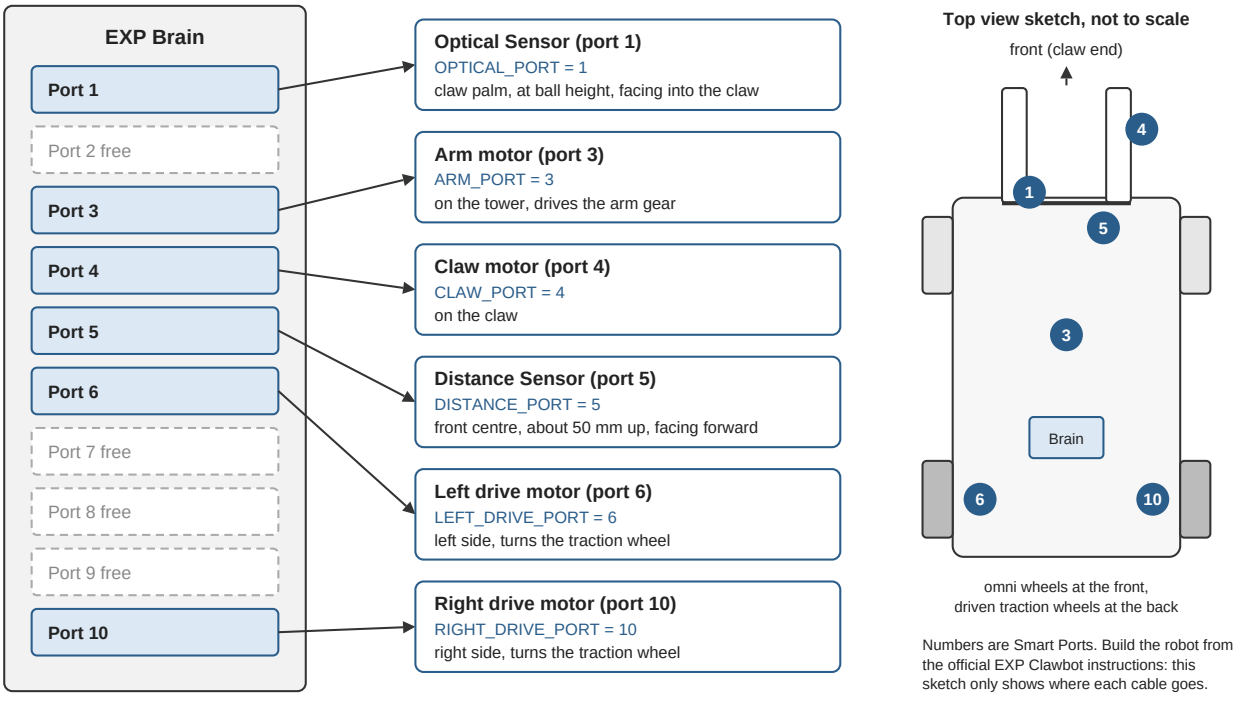
The robot knows where it is by dead reckoning: it adds up how far the wheels have turned along the heading from the Brain's gyro. Nothing ever corrects that estimate, so it drifts over a run. See Limitations.

Parts list

- A VEX EXP Clawbot, built from the official EXP Clawbot instructions (revision 3, listed on the EXP build instructions page). That build includes the EXP Brain, the battery and four Smart Motors: left drive, right drive, arm and claw.
- One VEX EXP Distance Sensor and one VEX EXP Optical Sensor, with a Smart Cable each, and kit parts to mount them.
- Optional: an SD card for the Brain, to keep the colour calibration. It must be FAT32 and 32 GB or smaller (SD Card page).
- A computer with VEXcode EXP and a cable to download the program.
- The field: the 300 mm grid board, three boxes 10 in by 10 in (red, green, blue), a fourth box or a marked spot for rejects, and balls about 15 cm across.

Wiring

Clawbot wiring: which Smart Port each motor and sensor uses



Which Smart Port each motor and sensor uses

Constant	Port	Device	Mount location	Facing	Height
`LEFT_DRIVE_PORT`	6	Left drive motor	Left side of the chassis, as in the build	-	-
`RIGHT_DRIVE_PORT`	10	Right drive motor	Right side of the chassis, as in the build	-	-
`ARM_PORT`	3	Arm motor	On the tower, driving the arm gear	-	-
`CLAW_PORT`	4	Claw motor	On the claw	-	-
`DISTANCE_PORT`	5	Distance Sensor	Front of the chassis, centred, looking out through the open claw	Forward, level	About 50 mm above the floor (MEASURE)
`OPTICAL_PORT`	1	Optical Sensor	Claw palm, centred	Into the claw	Middle of a held ball, about 75 mm up with the arm down (MEASURE)

The motor ports are the ones the official build uses. The two sensors are extras for this project, so their ports and mounts are my choice. Ports 2, 7, 8 and 9 are free. If you plug something into a different port, change the constant in `config/settings.py` to match.

Setup

On the robot:

1. Build the EXP Clawbot from the official instructions and mount the two sensors as in the wiring table.
2. Install VEXcode EXP (Install VEXcode for VEX EXP).
3. Make a new VEXcode EXP Python project, delete the template code, and paste in all of brain/clawbot.py. If the Brain says the program is too big, use brain/clawbot_compact.py instead: the same program without the comments.
4. Download it to the Brain.
5. Work through the bring-up checklist before anything else. A fresh copy is set to `RUN_MODE = "sensor_check"`, which never moves a motor.

On a laptop (only needed to change the code or run the tests), with Python 3.9 or newer:

```
python3 -m pip install -r requirements-dev.txt
```

After changing anything in `config/` or `src/`, rebuild the Brain files and paste the new one into VEXcode:

```
python3 tools/build_brain.py
```

Calibration

Three things need measuring on your own robot. The bring-up checklist walks through each.

1. **Arm and claw positions** (step 1b). The sensor check shows the arm and claw positions in motor degrees while you move them by hand. These go in `ARM_GRAB_DEG`, `ARM_CARRY_DEG`, `ARM_DROP_DEG` and `CLAW_EMPTY_CLOSED_DEG`.
2. **Colours** (step 1c). `RUN_MODE = "colour_calibrate"` records 20 Optical Sensor readings with the claw empty, then with each ball. Hue is 0 to 359 and brightness is a percent (Optical Sensor page). Each colour's reference hue is the average of its readings round the colour wheel, and its tolerance is three times how far they spread. The brightness gate sits halfway between the empty claw and the darkest ball. The result is saved to the SD card and printed in the VEXcode console as lines to paste into `config/settings.py`. Until you calibrate, the robot uses the middles of the hue ranges VEX gives for the sensor's own `color()` method.
3. **Driving** (step 3). Check the robot drives the distance it should (`WHEEL_TRAVEL_MM`) and turns without swinging past the heading (`TURN_KP`).

To check a colour log on a laptop, copy `colour_log.csv` off the SD card (or the console lines into a file) and run:

```
python3 tools/fit_colours.py colour_log.csv
```

It works out the same references the Brain does and shows how each logged reading would be classified. Put the file in `tests/data/recorded/` and the tests will check it too.

Running it

RUN_MODE	What happens	Motors move?
`"sensor_check"`	Live readings on the screen: arm and claw positions, heading, distance, hue, brightness and colour	Never
`"colour_calibrate"`	Records the empty claw and each ball, saves the colour references	Never
`"motor_check"`	After CHECK, spins each motor slowly for one second, one at a time	Yes, with the wheels off the ground
`"path"`	After CHECK, drives `SELECTED_PATH`	Yes
`"sort"`	After CHECK, homes the arm and claw, then sorts along `SORT_PATH`	Yes

To start, run the program from the Brain and keep the robot still while the gyro calibrates. In the modes that drive, the screen then says `CHECK = start`. Press CHECK and take your hand away: the robot waits one second before it moves. To stop, press Left or Right on the Brain at any time.

For the sort, put the balls on the west and north sides of `sort_loop` (north up line A, east along row 6), on the line and at least 450 mm from a corner. Keep the east and south sides clear, because the robot crosses them on its way to the boxes. The boxes go where `BOX_POSITIONS` and `REJECT_POSITION` say: the colour boxes off the east edge of the board and the reject spot off the south edge.

To see a sort without a robot:

```
python3 sim/run_sim.py --mode sort --demo-balls --plot
```

Changing the path

Paths live in `config/settings.py`. Headings are compass degrees: 0 points up the board towards row 6, 90 towards column D, and they go clockwise, like the Brain's gyro. There are two ways to write a path:

- As waypoints, like `ORIGINAL_WAYPOINTS`: a list of (x, y) points in mm. The robot turns to face each point and drives straight to it.
- As segments in `SEGMENT_PATHS`: `("turn", h)` turns on the spot to heading h, and `("drive", d)` drives d mm straight on (negative drives backwards).

For example, a 600 mm square:

```
SEGMENT_PATHS = {
    "square": [("drive", 600), ("turn", 90), ("drive", 600), ("turn", 180),
               ("drive", 600), ("turn", 270), ("drive", 600), ("turn", 0)],
}
SELECTED_PATH = "square"
```

`SELECTED_PATH` is the path `RUN_MODE = "path"` drives, and `SORT_PATH` is the one the sort searches along. After a change, look at it in the simulator and run the tests. They check that every path stays at least `BOX_CLEARANCE_MM` from every box, including while the claw swings round in a turn.

```
python3 sim/run_sim.py --path square --plot
```

A path for sorting needs long, straight sides that are well apart, because the robot only sees straight ahead.

Config reference

Every number the robot uses is in config/settings.py, and docs/CONFIG.md lists each one with its comment. It is generated by `tools/make_docs.py`. A test fails if `src/` contains a tuning number of its own. docs/ASSUMPTIONS.md says where each value came from, and which ones are estimates to measure.

The settings people change most:

Setting	What it does
`RUN_MODE`	Which of the five modes runs
`SELECTED_PATH`, `SORT_PATH`	Which path the drive and the sort use
`BOX_POSITIONS`, `REJECT_POSITION`	Where the boxes are, in mm from where line A crosses line 1
`COLOUR_REFERENCES`, `COLOUR_GATE_BRIGHTNESS`	The colour calibration, if there is no SD card
`ARM_*_DEG`, `CLAW_EMPTY_CLOSED_DEG`	Arm and claw positions, measured in bring-up step 1b
`DRIVE_MAX_PCT`, `TURN_MAX_PCT`	Top speeds, kept slow on purpose
`UNKNOWN_COLOUR_ACTION`	`"reject"` to drop unknown colours at the reject spot, `"stop"` to stop

Simulator and tests

```
python3 -m pytest
```

About 250 tests run in a few seconds. They cover the maths, the paths and box clearance, the settings, the Distance Sensor filter, the colour rule on sample readings, the calibration, full simulated sorts with every failure path, and the Brain file itself. The Brain file is checked for Python 3.4 syntax (the EXP Brain runs MicroPython 1.13, based on Python 3.4, MicroPython Libraries page) and is run against a stand-in for the `vex` module.

The simulator (sim/sim_hardware.py) has the same methods as the real hardware, so the same robot code runs in it. It is simple on purpose, and passing it does not mean the robot works:

- the sensor values in it, and in `tests/data/`, are made up, not recorded
- it uses the same geometry settings as the robot, so it cannot catch a wrong measurement
- balls never roll, the arm has no weight, and the Distance Sensor is a single straight ray
- wheel slip and gyro drift are off unless a test turns them on

Troubleshooting

What you see	What to check
The program will not load, or a memory error	Use `brain/clawbot_compact.py`
`Bad RUN_MODE`, `Bad path setting`, `Box on the board` or `Bad unknown rule`	The setting named on the screen in `config/settings.py`
`Gyro calibration timed out`	Keep the robot still while it calibrates, then restart
`Not started.`	CHECK was not pressed within two minutes
A wheel, the arm or the claw moves the wrong way	Its `*_REVERSED` setting. Use the motor check
A motor check number is 0	That motor's cable and port
The robot drives a few percent too far or too short	`WHEEL_TRAVEL_MM`
Turns swing past the heading and back	Lower `TURN_KP`
`Dist` sees something with nothing in front	The Distance Sensor is seeing the claw: move it, or raise `ARM_SEARCH_DEG`
`Colour` is always `none`	The ball is not reaching the sensor, or the brightness gate is too high: calibrate again
`Colour` is always `unknown`	Calibrate again in the light the robot will sort in
`SORT STOPPED`	The second line says why. The table at the end of the bring-up checklist says what to check

Limitations

Dead reckoning drifts. Nothing outside the robot ever corrects its position estimate, so wheel slip and gyro drift build up over a run. In the simulator, with one ball of each colour and one unknown:

Gyro drift	Result
none	all 4 balls land in the right place
0.6 degrees a minute	all 4 land
1.2 degrees a minute	the robot counts all 4 as sorted, but one lands beside its box
3 degrees a minute	the robot hits a box on the way in and stops

Nothing checks what lands in a box. The robot counts a ball as sorted when the claw is empty after it opens over the box. It cannot see whether the ball went in.

It only sees straight ahead. There is one Distance Sensor, facing forwards. The API gives its range as 20 mm to 2000 mm (Distance Sensor page), and this program only reacts to objects within 400 mm. A ball beside the path is not seen, and one within about 22 cm of where the robot drives can be clipped. When the robot turns on the spot, nothing watches the sides.

Anything on the board is taken for a ball first. A box or a shoe on the board gets approached. The robot only gives up when the claw meets it and nothing is in the claw.

A colour that was never calibrated can pass as one that was, if its hue happens to be close. Asking five of seven readings to agree makes that less likely, not impossible.

The claw has not been tried with the balls. Nobody has checked yet that a 15 cm ball fits in the EXP claw. The arm and claw positions in `config/settings.py` are guesses until they are measured.

Safety

- Speeds are slow by default: driving at most 30 % power, turning 25 %, the arm 40 %. A test fails if they are raised past 35, 30 and 50 %.
- The Left and Right buttons on the Brain stop every motor at any time.
- A fresh copy starts in `sensor_check`, which never moves a motor.
- After CHECK the robot waits one second, so your hand is clear.
- Keep fingers away from the claw and the arm while the program runs. The claw grips with motor power, and the arm can pinch.
- Every wait and every move has a time limit, and a drive stops by itself when the wheels stall against something.
- Follow the bring-up order: sensors first, motors with the wheels off the ground, slow driving, then the full sequence.
- Watch the robot all the time it runs, keep people and pets off the field, and never run it on a table.
- The simulator and the tests passing does not make the robot safe. Only the bring-up checks on the real robot can show that.

Licence and credits

There is no licence yet. One will be added before the repository is made public.

- The EXP Clawbot design, its build instructions and the VEX EXP Python API are VEX Robotics' work. This project links to them (docs/REFERENCES.md) and does not copy them.
- The `vex` module the program imports on the Brain is VEX's software, under the VEXcode licence. It is not part of this repository.
- On the laptop: Python, pytest, matplotlib, ReportLab and svglib.
- The program this project started from is kept unchanged on the `submission-v1` branch and tag.

The PDF guide is built from this README and the other docs with `python3 tools/build_guide.py`.

Not affiliated with VEX

This is a personal student project. It is not affiliated with or endorsed by VEX Robotics. VEX and VEX Robotics are trademarks of Innovation First, Inc. (copyright notice).

Bring-up checklist

UNTESTED ON HARDWARE. This code has not been run on a real robot yet. This checklist is the order to try it in the first time, so that a wrong setting shows up while the robot cannot hurt itself or anyone.

The order is: sensors alone, then motors with the wheels off the ground, then slow driving, then the full sequence. Do not skip ahead. A fresh copy is set to `RUN_MODE = "sensor_check"`, which never moves a motor.

How to stop the robot

- Press the **Left** or **Right** button on the Brain. The program stops every motor and shows STOPPED.
- Pick the robot up if it is about to hit something.
- [VERIFY] Find out how VEXos ends a running program on your Brain (for example the X button) before step 3, so you have a second way to stop it.

Changing a setting

Each step changes `RUN_MODE`, and steps 3 and 4 also choose a path (`SELECTED_PATH` or `SORT_PATH`). Either:

- change it in `config/settings.py`, run `python3 tools/build_brain.py` and paste the new `brain/clawbot.py` into VEXcode (this keeps the repository correct), or
- change the same line near the top of the pasted file in VEXcode, which is quicker while testing. Copy the change back to `config/settings.py` afterwards.

Before switching on

[ ] The robot is built as in the EXP Clawbot build instructions (rev 3).

[ ] Left drive motor in port 6, right drive motor in port 10, arm motor in port 3, claw motor in port 4.

[ ] Distance Sensor in port 5, low at the front centre, facing forward.

[ ] Optical Sensor in port 1, on the claw, facing into the claw.

[ ] Battery charged.

[ ] Optional: an SD card in the Brain, FAT32 and 32 GB or smaller, to keep the colour calibration between runs.

Step 1: sensors alone

No motor moves in either part of this step.

1a. Live readings

Setting: `RUN_MODE = "sensor_check"`.

1. Put the robot on the board and start the program. Keep it still while the screen says "Calibrating gyro".
2. The screen then shows:
   - `Arm` and `Claw`: their positions in motor degrees, counted from where they were when the program started
   - `Head`: the gyro heading
   - `Dist`: the filtered distance in mm, with `IN` instead of `mm` when something is within `OBJECT_RANGE_MM` (400 mm), or `none`
   - `Hue` and `Br`: the raw Optical Sensor hue and brightness
   - `Colour`: the colour rule's answer for the last few readings: a colour, `none` (nothing in the claw) or `unknown`

[ ] The program starts at all. `brain/clawbot.py` is a big file (about 80 KB). [VERIFY] If VEXcode or the Brain reports a size or memory error, write down the exact message and try `brain/clawbot_compact.py`, the

same program without comments.

[ ] Turn the robot by hand about a quarter turn clockwise. The heading goes up by about 90.

[ ] Hold a hand about 300 mm in front of the Distance Sensor. It shows roughly `Dist 300 IN`. At about 600 mm it shows `Dist 600 mm`. Take the hand away and it shows `none` or a much bigger number.

[ ] Put a ball in the claw. Brightness goes up and the hue changes with the ball's colour. Until step 1c is done the colour comes from the VEX default hue ranges, so it may be wrong.

[ ] With the claw empty, `Colour` shows `none`.

[ ] With nothing in front of the robot, the arm down and the claw open, `Dist` shows `none`. If it sees the claw, move the Distance Sensor or raise `ARM_SEARCH_DEG` until it does not.

[ ] Press Left or Right. The screen shows STOPPED.

1b. Arm and claw positions

Still `RUN_MODE = "sensor_check"`. Before starting the program, move the arm all the way down and open the claw fully by hand. Both then read 0. Move them slowly by hand and write down the `Arm` and `Claw` numbers for:

[ ] `ARM_GRAB_DEG`: the claw at the height to grip a ball on the floor (usually 0)

[ ] `ARM_CARRY_DEG`: a held ball clear of the floor and out of the Distance Sensor's view (`Dist` no longer sees it)

[ ] `ARM_DROP_DEG`: the bottom of a held ball above the rim of a box

[ ] `CLAW_EMPTY_CLOSED_DEG`: the claw closed with nothing in it

[ ] the `Claw` number with a ball held. It must be at least `CLAW_GRIP_MARGIN_DEG` (20) below `CLAW_EMPTY_CLOSED_DEG`, or the robot cannot tell a held ball from an empty claw

[ ] `DROP_REACH_MM`: with the arm at `ARM_DROP_DEG`, the distance along the floor from the middle of the traction wheel axle to the middle of the held ball

Put these in `config/settings.py`. [VERIFY] The arm motor may be hard to turn by hand through its gears. If it will not move, keep the settings as they are for now: step 4b shows whether the grab, carry and drop heights work, and they can be changed from there.

1c. Colour calibration

Setting: `RUN_MODE = "colour_calibrate"`. Have the red, green and blue balls ready. The room light should be the same as when the robot will sort.

1. Start the program and keep the robot still for the gyro.
2. The screen says `Claw empty, then press CHECK`. Check the claw is empty, press CHECK and keep still for about a second while it reads.
3. It then asks for each ball in turn (`red in claw` and so on). Put the ball where it sits when the claw holds it, press CHECK, keep it still.
4. The last screen shows `Saved to SD` (or `No SD: console`), then each colour's reference hue and tolerance, then the brightness gate.

[ ] Every colour shows a hue, and the screen does not say `CLOSE!`. `CLOSE!` means two colours look alike to the sensor: check the lighting and the balls.

[ ] If it says `CALIBRATE FAILED` and `red too dark` (or another colour), that ball was not in front of the sensor, or the LED is off.

[ ] Run 1a again. Each ball now shows its colour, and the empty claw shows `none`.

[ ] Keep the readings. They are in the VEXcode console as `label,hue,brightness` lines, and in `colour_log.csv` on the SD card. On a laptop, `python3 tools/fit_colours.py colour_log.csv` shows how each reading would be classified. Copy the file into `tests/data/recorded/` and the tests will check real data as well.

Without an SD card the calibration is not kept. Paste the `COLOUR_REFERENCES` and `COLOUR_GATE_BRIGHTNESS` lines from the console into `config/settings.py` and rebuild instead. [VERIFY] What `loadfile` does when the file is missing is not documented. The program treats an error or an empty file as "no calibration" and uses `settings.py`.

Step 2: motors with the wheels off the ground

Setting: `RUN_MODE = "motor_check"`.

1. Stand the robot on a box so both drive wheels hang in the air.
2. Move the arm by hand to about halfway up and open the claw halfway, so each has room to move either way.
3. Start the program, wait for "CHECK = start", press CHECK and take your hand away. After one second each motor spins for one second at 20 % power, one at a time.

[ ] Left wheel rolls forward (towards the claw end).

[ ] Right wheel rolls forward.

[ ] Arm goes up.

[ ] Claw closes.

[ ] The last screen shows four numbers above 0. A 0 means that motor did not turn: check its cable and port.

If a part moved the wrong way, flip its `*_REVERSED` setting and do this step again before going on.

Step 3: slow driving

Settings: `RUN_MODE = "path"`, `SELECTED_PATH = "bringup_line"`.

The robot drives 300 mm forward, turns round, drives back and turns to face the way it started. Leave about 1 m of clear space around it.

1. Mark the start position on the board with tape.
2. Start the program, press CHECK, take your hand away.

[ ] It drives forward, turns round, comes back and stops.

[ ] Measure how far the end position is from the tape and write it down.

[ ] If it turns too far and swings back, lower `TURN_KP`. If it stops just short of a heading and hums, raise `TURN_MIN_PCT` a little.

[ ] If it drives a few percent too far or too short, measure the wheel and check `WHEEL_TRAVEL_MM`.

Step 4: the full sequence

4a. The whole path

Settings: `RUN_MODE = "path"`, `SELECTED_PATH = "original"`.

The start point is on line A, 150 mm before line 1, facing along line A towards row 6 (x 0 mm, y -150 mm in `config/settings.py`).

[ ] The robot follows the route and comes back to the start.

[ ] Measure the end error and compare it with what the screen says. Expect them to differ: dead reckoning drifts, and the robot cannot see its own error.

[ ] The claw never comes near the boxes.

4b. Sort one ball

Settings: `RUN_MODE = "sort"`, `SORT_PATH = "sort_loop"`. Put the boxes where `BOX_POSITIONS` and `REJECT_POSITION` say. Put one red ball on line A, 450 mm in front of the start point.

1. Start the program, press CHECK, take your hand away. The screen says whether the colours came from the SD card or from `settings.py`.
2. The robot first homes the arm and claw: the arm goes gently down onto its stop and the claw opens onto its stop.

[ ] It stops when it sees the ball, creeps up to it and closes the claw.

[ ] The screen shows `CLASSIFY`, then `Colour red`.

[ ] It drives to the line-up point in front of the red box, raises the arm, moves in, and lets go. Check the ball really lands in the box.

[ ] It backs out, lowers the arm, goes back to where it saw the ball and carries on round the loop.

[ ] At the end the screen shows `SORT DONE` and `R1 G0 B0 Rej 0`.

4c. Sort all the balls

Put one ball of each colour, and one ball of a colour you did not calibrate, on the west and north sides of the loop, on the line and at least 450 mm from a corner. Keep the east and south sides clear.

[ ] Each ball goes to its own box and the odd one goes to the reject spot.

[ ] Note any ball that lands beside a box instead of in it. The robot cannot tell, so only you can check this.

If the screen says `SORT STOPPED`, the second line says why:

Message	What to check
`arm not homed`, `claw not homed`	The motor did not reach its stop within the time: cable, port, or `*_REVERSED`
`arm timed out`, `claw timed out`	A move did not finish: something in the way, or `ARM_MIN_PCT` too low
`drive blocked`	The wheels stopped turning while driving: something is in the way
`turn timed out`, `drive timed out`	A turn or drive took too long: low battery, or the gains need tuning
`path blocked`, `way blocked`	Something was in the way and CHECK was not pressed after clearing it
`unknown colour`	Only when `UNKNOWN_COLOUR_ACTION = "stop"`
`box not reached`	The last move into the box did not finish: the chassis may have hit the box
`ball stuck`	The ball stayed in the claw after it opened twice
`took too long`	The whole sort passed `MISSION_TIMEOUT_MS`

Results log

Copy this table into your notes and fill in a row each time.

Date	Step	Pass or fail	What happened	Settings changed

Assumptions

Where each value in `config/settings.py` comes from, and what still has to be checked on the real robot. Nothing in this project has run on a robot yet, so all of it is **UNTESTED ON HARDWARE** until the steps in BRINGUP.md have been done.

What the status words mean:

Status	Meaning
From VEX	Taken from an official VEX page or build instruction, linked below
Given	Told to me by the person who built the robot
Chosen	A project decision. Change it if something else suits your setup
MEASURE	An estimate. Measure it on your robot and update the setting
VERIFY	Believed to be true but not yet confirmed from an official source

Robot

Setting	Value	Status	Where it comes from
Robot	EXP Clawbot	Given	Built from the EXP Clawbot build instructions, rev 3 [1]
`LEFT_DRIVE_PORT`	6	From VEX	Build instructions, step 14 [1]
`RIGHT_DRIVE_PORT`	10	From VEX	Build instructions, step 14 [1]
`ARM_PORT`	3	From VEX	Build instructions, step 41 [1]
`CLAW_PORT`	4	From VEX	Build instructions, step 41 [1]
`DISTANCE_PORT`	5	Chosen	The port the Distance Sensor used in the assignment program
`OPTICAL_PORT`	1	Chosen	A free Smart Port. The stock Clawbot has no sensors
`RIGHT_DRIVE_REVERSED`	True	VERIFY	The right motor faces the other way. The motor check confirms it
`ARM_REVERSED`, `CLAW_REVERSED`	False	MEASURE	Positive power must raise the arm and close the claw. The motor check confirms it
`WHEEL_TRAVEL_MM`	260	From VEX	The build uses 3.25 in wheels, listed as 260 mm travel [1]
`DRIVE_GEAR_RATIO`	1.0	From VEX	The drive motors turn the traction wheels directly [1]
`TRACK_WIDTH_MM`	240	MEASURE	Left wheel centre to right wheel centre. Only the simulator uses it
`FRONT_MM`, `REAR_MM`, `ROBOT_WIDTH_MM`	330, 80, 290	MEASURE	Robot outline measured from the turning centre. Used for box clearance
`SENSOR_OFFSET_MM`	200	MEASURE	Turning centre to the Distance Sensor face
`CHASSIS_FRONT_MM`	200	MEASURE	Turning centre to the front of the chassis under the claw. Must stay out of a box at the drop point
`CLAW_PALM_MM`	230	MEASURE	Turning centre to the back of the claw opening. Only the simulator uses it
Turning centre	middle of the traction wheel axle	From VEX	The omni wheels at the claw end are not driven [1]
Sensor mounting	Distance Sensor low at the front centre, looking out through the open claw. Optical Sensor on the claw palm, facing into the claw	Chosen	With the arm down and nothing in front, the Distance Sensor must not see the claw (bring-up step 1a)

Field and boxes

Setting	Value	Status	Where it comes from
`GRID_MM`	300	Given	Each board square is 300 mm by 300 mm
Board edges	x -300 to 1200 mm, y -300 to 1500 mm	Given	Same board as the assignment
`BALL_DIAMETER_MM`	150	Given	The balls are about 15 cm
`BOX_SIZE_MM`	254	Given	The boxes are 10 in by 10 in
`BOX_POSITIONS`	x 1550 mm, at y 1200, 600 and 0	Chosen	Off the east edge. The original path uses almost the whole board
`REJECT_POSITION`	x 450, y -650	Chosen	Off the south edge, so its approach lane does not cross the sides where balls go
`BOX_CLEARANCE_MM`	100	Chosen	A test checks the robot outline stays this far from every box on every path. The closest point in the simulator is 164 mm on the original path and 132 mm on `sort_loop`
`SORT_PATH`	`sort_loop`	Chosen	A loop round the board with long, well separated sides. The original path's lines are only 150 to 300 mm apart, too close to leave balls beside them
`START_X`, `START_Y`, `START_HEADING`	0, -150, 0	Given	The start of the assignment route
`ORIGINAL_WAYPOINTS`	14 points	Given	Copied exactly from the assignment program. A test compares them with the `submission-v1` tag

Software and API

Item	Status	Where it comes from
The EXP Brain runs MicroPython 1.13, based on Python 3.4	From VEX	MicroPython Libraries page [2]. The build script rejects newer syntax
A VEXcode EXP Python project is a single file	VERIFY	The reason `tools/build_brain.py` joins everything into `brain/clawbot.py`
Brain buttons a program can read: Check, Left, Right	From VEX	Brain buttons page [3]
Distance Sensor range 20 to 2000 mm	From VEX	Distance Sensor page [4]
How often the Distance Sensor gives a new reading	VERIFY	Not on the API page. `DISTANCE_SAMPLE_MS` is 50 ms, a guess
The gyro is built into the EXP Brain and needs no port	From VEX	Inertial page [5]
Optical Sensor: `hue()` is 0 to 359, `brightness()` is a percent, `set_light_power()` takes whole percent and 0 turns the LED off	From VEX	Optical Sensor page [6]
Optical Sensor `color()` hue ranges: red 340 to 20, green 80 to 140, blue 200 to 240	From VEX	Optical Sensor page [6]. The default `COLOUR_REFERENCES` are the middles of these
SD card: FAT32, 32 GB or smaller, files in the root folder, names with an extension, `savefile` and `loadfile` take and give a `bytearray`	From VEX	SD Card page [7]
What `loadfile` does when the file is missing	VERIFY	Not on the page. The code treats an error or an empty file as "no calibration"
The Brain screen fits five lines of about 16 characters	VERIFY	The assignment program's screen used five lines of up to 16 characters
Motor calls in `src/hardware.py`	From VEX	Motor page [8]
Screen, Sound, `wait` and timer calls	From VEX	Screen [9], Sound [10], Control [11] and Timer [12] pages

Sensing

Setting	Value	Status	Notes
`DISTANCE_WINDOW`, `DISTANCE_MIN_VALID`	5, 3	Chosen	The distance is the median of the last 5 readings, and needs at least 3 of them valid
`OBJECT_RANGE_MM`	400	Chosen	While following the path, an object this near that lies on the board starts an approach
`COLOUR_REFERENCES`	red 0 +-20, green 110 +-30, blue 220 +-20	From VEX	Only a starting point. The calibration replaces them
`COLOUR_GATE_BRIGHTNESS`	10	MEASURE	The calibration replaces it with a value halfway between the empty claw and the darkest ball
`COLOUR_READS`, `COLOUR_MIN_VOTES`	7, 5	Chosen	A colour needs 5 of 7 readings to agree, otherwise the answer is "unknown"
`COLOUR_SPREAD_FACTOR` and the tolerance limits	3, 8 to 30 degrees	Chosen	Tolerance is 3 times the spread of the calibration hues, kept between 8 and 30 degrees
`COLOUR_CAL_SAMPLES`, `COLOUR_CAL_MIN_GOOD`	20, 15	Chosen	20 readings per step, and at least 15 bright enough for each ball
Sample readings in `tests/data/`		SYNTHETIC	The `*_synthetic.csv` files are made up, not recorded. Real logs go in `tests/data/recorded/`

Speeds and control

Setting	Value	Status	Notes
`DRIVE_MAX_PCT`	30	Chosen	Kept slow on purpose. A test fails if it goes above 35
`TURN_MAX_PCT`	25	Chosen	Kept slow on purpose. A test fails if it goes above 30
`DRIVE_MIN_PCT`, `TURN_MIN_PCT`	7, 6	MEASURE	The lowest power that still moves the robot
`DRIVE_KP`, `STEER_KP`, `TURN_KP`, `TURN_KD`	0.3, 1.0, 0.8, 2.0	Chosen	Only tried in the simulator. Expect to retune on the robot
`START_DELAY_MS`	1000	Chosen	Same one-second pause as the assignment program
Timeouts	see `settings.py`	Chosen	Every wait has one. A test fails if any is 0 or less
`DRIVE_STALL_MM`, `DRIVE_STALL_MS`	5, 500	Chosen	A drive that moves less than 5 mm in half a second is pushing on something, so it stops
`OPTICAL_LIGHT_PCT`	50	MEASURE	LED power for the colour readings, a whole number as the API asks. Calibrate again after changing it

Arm, claw and sorting

Setting	Value	Status	Notes
Homing	arm down, claw open, onto their stops	Chosen	There are no limit switches, so the ends of travel are found by stalling at low power (15 %)
`STALL_DEG`, `STALL_MS`	3, 300	Chosen	Turning less than 3 degrees in 300 ms counts as reaching a stop
`ARM_SEARCH_DEG`, `ARM_GRAB_DEG`	0, 0	MEASURE	Claw down at ball height. Bring-up step 1b
`ARM_CARRY_DEG`	150	MEASURE	High enough that the Distance Sensor no longer sees the held ball
`ARM_DROP_DEG`	400	MEASURE	The bottom of the ball above the box rim
`CLAW_EMPTY_CLOSED_DEG`, `CLAW_GRIP_MARGIN_DEG`	120, 20	MEASURE	A claw that stops 20 degrees short of fully shut, while the Optical Sensor sees something, is holding a ball
`DROP_REACH_MM`	310	MEASURE	Turning centre to the held ball's centre with the arm up
`DROP_INSET_MM`	90	Chosen	The ball's centre this far inside the box's near edge: 15 mm more than its radius
`APPROACH_STOP_MM`, `CREEP_MAX_MM`, `IN_CLAW_READS`	100, 150, 3	Chosen	Drive up until the ball is 100 mm from the Distance Sensor, then creep until 3 bright readings in a row
`OBSTACLE_STOP_MM`	150	Chosen	Anything this near that is not a ball to fetch stops the robot
`IGNORE_RADIUS_MM`	200	Chosen	After giving up on an object, detections this near it are ignored
`BOX_STAGING_MM`	400	Chosen	Line up this far from the drop point, and back out to it afterwards
`UNKNOWN_COLOUR_ACTION`	`"reject"`	Chosen	Or `"stop"`, which the brief also allows
`GRAB_TRIES`, `RELEASE_TRIES`, `NAV_TRIES`	2, 2, 3	Chosen	
`MISSION_TIMEOUT_MS`	15 minutes	Chosen	A whole sort in the simulator takes about 3 minutes

Simulator only

These describe the simulator's model, not the robot. They live in `sim/sim_hardware.py` and never go onto the Brain.

Value	Status	Notes
Motor speed at 100 % power: 127 rpm	VERIFY	Only changes how long simulated runs take
Motor response lag: 60 ms	Chosen	A rough guess
Gyro calibration: 2 s	Chosen	The real calibration time may differ. The program allows 5 s
Wheel slip and gyro drift	Chosen per test	Off by default. Tests turn them on to show the drift
Ball hues: red 3, green 118, blue 215, orange 32 degrees. White has no clear hue	Chosen	Guesses for the model, not measurements
Empty-claw brightness 4 %, hue noise 4 degrees, brightness noise 2 %	Chosen	Guesses for the model
Distance Sensor misses and wild readings	Chosen per test	Off by default. Tests turn them on to check the filter
The claw stops at 60 degrees on a ball and shuts at 120 without one	Chosen	Model values
A ball can be gripped if its centre is within 60 mm of the claw's centre line	Chosen	Model value
The Optical Sensor sees a floor ball within 10 mm of the palm	Chosen	Model value

Value	Status	Notes
The arm clears the Distance Sensor above 100 degrees and passes over a box above 300	Chosen	Model values
Balls never roll	Chosen	A real ball may roll away when the claw touches it. The grab retry is there for that, but the simulator never tests it

Open risks

1. **Will a 15 cm ball fit in the claw?** Still not checked. Open the claw fully and try a ball by hand before any sort. If it does not fit, the pick-up has to change (smaller balls, or a different grip). The simulator assumes it fits.
2. **Dead reckoning drifts.** See the README. The robot cannot notice its own position error.
3. **Gains tuned only in the simulator.** The real robot will behave differently. The bring-up checklist says what to change first.
4. **Colours that were not calibrated.** A ball of another colour can be read as a calibrated colour if its hue is close enough. In the synthetic samples, 3 of 14 single readings of a white ball looked blue. The vote (5 of 7 must agree) caught it, but cannot rule it out.
5. **Optical Sensor on the claw.** The LED may reflect off the claw itself. The empty-claw calibration step measures that, but only with the claw where it was during calibration.
6. **Drops can miss without the robot knowing.** In the simulator a gyro drift of about 1 degree a minute is enough for one of four balls to land beside its box, while the robot counts it as sorted.
7. **It only sees straight ahead.** A ball beside the path is not seen, and one within about 22 cm of where the robot drives can be clipped, including while it turns on the spot.
8. **Geometry is guessed.** The arm angles, claw angles and reach are placeholders until they are measured in bring-up step 1b.

Links

1. EXP Clawbot build instructions, rev 3: <https://link.vex.com/exp/builds/clawbot> (listed on <https://www.vexrobotics.com/exp/downloads/build-instructions>)
2. MicroPython Libraries: <https://api.vex.com/exp/home/python/Micropython_libraries.html>
3. Brain (buttons): <https://api.vex.com/exp/home/python/Sensing/Brain.html>
4. Distance Sensor: <https://api.vex.com/exp/home/python/Sensing/distance-sensor.html>
5. Inertial: <https://api.vex.com/exp/home/python/Inertial.html>
6. Optical Sensor: <https://api.vex.com/exp/home/python/Sensing/optical-sensor.html>
7. SD Card: <https://api.vex.com/exp/home/python/SDcard.html>
8. Motor: <https://api.vex.com/exp/home/python/Motion/motor_and_motor_group.html>
9. Screen: <https://api.vex.com/exp/home/python/Screen.html>
10. Sound: <https://api.vex.com/exp/home/python/Sound.html>
11. Control (`wait`): <https://api.vex.com/exp/home/python/Logic/Control.html>
12. Timer: <https://api.vex.com/exp/home/python/Logic/Timer.html>

The VEX documents are linked, not copied into this repository. REFERENCES.md lists every VEX page the project uses.

Settings reference

Generated from `config/settings.py` by `tools/make_docs.py`. Do not edit this file: change the comments in `settings.py` and run the script again. Values marked MEASURE are estimates to check on your own robot, and ASSUMPTIONS.md says where each value came from.

Smart Ports

Motor ports follow the official EXP Clawbot build instructions (rev 3): step 14 wires the drive motors, step 41 wires the arm and the claw.

Setting	Value	Notes
`LEFT_DRIVE_PORT`	`6`	
`RIGHT_DRIVE_PORT`	`10`	
`ARM_PORT`	`3`	
`CLAW_PORT`	`4`	
`DISTANCE_PORT`	`5`	The stock Clawbot has no sensors. These two are added for this project. Same port the Distance Sensor used in the earlier program.
`OPTICAL_PORT`	`1`	A free port chosen for the Optical Sensor.
`LEFT_DRIVE_REVERSED`	`False`	
`RIGHT_DRIVE_REVERSED`	`True`	The right motor is mounted facing the other way.
`ARM_REVERSED`	`False`	MEASURE: positive power must raise the arm.
`CLAW_REVERSED`	`False`	MEASURE: positive power must close the claw.

Drive geometry

The drive motors turn the two 3.25 in traction wheels directly. The omni wheels at the claw end spin freely, so the robot pivots about the middle of the traction wheel axle. That point is the "turning centre" that the position tracking follows.

Setting	Value	Notes
`WHEEL_TRAVEL_MM`	`260.0`	3.25 in wheel, 260 mm per turn (build parts list).
`DRIVE_GEAR_RATIO`	`1.0`	Wheel turns per motor turn.
`TRACK_WIDTH_MM`	`240.0`	MEASURE: left wheel centre to right wheel centre.
`FRONT_MM`	`330.0`	Robot outline, measured from the turning centre. MEASURE: turning centre to the claw tips.
`REAR_MM`	`80.0`	MEASURE: turning centre to the back of the robot.
`ROBOT_WIDTH_MM`	`290.0`	MEASURE: widest point.
`SENSOR_OFFSET_MM`	`200.0`	MEASURE: turning centre to the Distance Sensor face.
`CHASSIS_FRONT_MM`	`200.0`	MEASURE: turning centre to the front of the chassis, under the claw.
`CLAW_PALM_MM`	`230.0`	MEASURE: turning centre to the back of the claw's opening.

The field

The 300 mm grid board. The origin is where line A crosses line 1. +x points east towards column D, +y points north towards row 6.

Setting	Value	Notes
`GRID_MM`	`300.0`	
`BOARD_MIN_X`	`-300.0`	
`BOARD_MAX_X`	`1200.0`	
`BOARD_MIN_Y`	`-300.0`	

Setting	Value	Notes
BOARD_MAX_Y	1500.0	
BALL_DIAMETER_MM	150.0	The balls being sorted are about 15 cm across.
BOX_SIZE_MM	254.0	The boxes are 10 in by 10 in.
COLOURS	["red", "green", "blue"]	The ball colours; each one has a box below.
BOX_POSITIONS	see config/settings.py	Box centres in mm. The original path uses almost the whole board, so the boxes sit off its edges: the colour boxes to the east, the reject spot to the south. Move them to suit your room. tests/test_paths.py checks that the robot outline on every path stays BOX_CLEARANCE_MM away from every box, including while the claw swings round in a turn.
REJECT_POSITION	(450.0, -650.0)	Where a ball of unknown colour is left.
BOX_CLEARANCE_MM	100.0	

Paths

The robot starts with its turning centre here, facing north.

Setting	Value	Notes
START_X	0.0	
START_Y	-150.0	
START_HEADING	0.0	
ORIGINAL_WAYPOINTS	see config/settings.py	"original" is the route from the assignment program, copied exactly. Each point is (x, y) in mm. The robot turns to face the next point and then drives straight to it. At the end it turns back to face north.
ORIGINAL_FINAL_HEADING	0.0	
SEGMENT_PATHS	see config/settings.py	Paths written directly as segments: ("turn", h) turn on the spot to compass heading h, in degrees ("drive", d) drive d mm straight along the current heading A negative d drives backwards. A loop round the board for sorting: north up line A, east along row 6, south down column D and back west. Put the balls on the west and north sides, on the line, at least 450 mm from a corner. Keep the east and south sides clear: the robot crosses them on its way to the boxes. It only sees straight ahead, so a ball beside its path is not seen, and one within about 22 cm of where it drives can be clipped.
SELECTED_PATH	"original"	The path RUN_MODE "path" drives.
SORT_PATH	"sort_loop"	The path RUN_MODE "sort" searches along.

Run mode

What the program does once the gyro has calibrated. The first time the code runs on a robot, go through the modes in the order docs/BRINGUP.md gives:

- "sensor_check" show live sensor readings; no motor ever moves
- "colour_calibrate" record the Optical Sensor with the claw empty and with each ball in turn, then save the colour references
- "motor_check" after CHECK, spin each motor slowly, one at a time
- "path" after CHECK, drive SELECTED_PATH
- "sort" after CHECK, search along SELECTED_PATH and sort the balls

The default is "sensor_check", so a fresh copy never drives off by itself.

Setting	Value	Notes
RUN_MODE	"sensor_check"	

Setting	Value	Notes
START_DELAY_MS	1000	Pause after CHECK so your hand is clear of the robot.

Speeds (slow)

Motor power is in percent. Defaults are deliberately slow.

Setting	Value	Notes
LOOP_MS	10	Every control loop runs once per LOOP_MS.
DRIVE_MAX_PCT	30.0	
DRIVE_MIN_PCT	7.0	MEASURE: lowest power that still moves the robot.
DRIVE_RAMP_PCT	2.0	Speed added per loop while speeding up.
DRIVE_KP	0.3	Percent per mm of distance still to go.
DRIVE_TOLERANCE_MM	5.0	
STEER_KP	1.0	Percent per degree of heading error.
STEER_MAX_PCT	10.0	
TURN_MAX_PCT	25.0	
TURN_MIN_PCT	6.0	MEASURE: lowest power that still turns the robot.
TURN_KP	0.8	Percent per degree of heading error.
TURN_KD	2.0	Percent per degree of change since the last loop.
TURN_TOLERANCE_DEG	1.5	
TURN_SETTLE_LOOPS	4	Loops in a row inside the tolerance before a turn ends.
STOP_SETTLE_MS	150	Time allowed for the robot to come to rest.
DRIVE_STALL_MM	5.0	A drive that moves less than this ...
DRIVE_STALL_MS	500	... in this long is stuck against something, so it stops.
MOTOR_CHECK_PCT	20.0	Power for each motor in the motor check.
MOTOR_CHECK_MS	1000	How long each motor spins in the motor check.
MOTOR_CHECK_GAP_MS	500	Pause after each motor in the motor check.

Timeouts

Setting	Value	Notes
IMU_CALIBRATION_TIMEOUT_MS	5000	
START_WAIT_TIMEOUT_MS	120000	Program ends if CHECK is not pressed in 2 minutes.
TURN_TIMEOUT_MS	6000	
DRIVE_TIMEOUT_BASE_MS	2000	
DRIVE_TIMEOUT_MS_PER_MM	15	
SENSOR_CHECK_TIMEOUT_MS	300000	The sensor check ends after 5 minutes.
HOME_TIMEOUT_MS	4000	
ARM_TIMEOUT_MS	4000	
CLAW_TIMEOUT_MS	2500	
APPROACH_TIMEOUT_MS	10000	
CREEP_TIMEOUT_MS	6000	
BLOCKED_TIMEOUT_MS	120000	How long "clear the way" waits for CHECK.
MISSION_TIMEOUT_MS	900000	A whole sort may take at most 15 minutes.

Sensors

Distance Sensor. object_distance() covers 20 mm to 2000 mm (VEX EXP Python API, Distance Sensor page). A reading outside that range, or one taken while the sensor says it sees nothing, is thrown away. src/distance.py explains the median and the "in range" rule.

Setting	Value	Notes
DISTANCE_MIN_MM	20.0	
DISTANCE_MAX_MM	2000.0	
DISTANCE_SAMPLE_MS	50	VERIFY: take a new reading at most this often.
DISTANCE_WINDOW	5	The median uses the last this-many readings.
DISTANCE_MIN_VALID	3	Valid readings the window needs before it counts.
OBJECT_RANGE_MM	400.0	An object at this distance or nearer is "in range".
OPTICAL_LIGHT_PCT	50	Optical Sensor. src/colour.py explains the colour rule. hue() is 0 to 359 degrees and brightness() is a percent (VEX EXP Python API, Optical Sensor). MEASURE: LED power, a whole number of percent.
COLOUR_READS	7	Readings taken for one colour decision.
COLOUR_MIN_VOTES	5	How many of them must agree, or the answer is "unknown".
COLOUR_READ_MS	30	Gap between those readings.
COLOUR_REFERENCES	see config/settings.py	Reference hues used until you run the colour calibration, as (hue, tolerance) in degrees. These starting values are the middles of the hue ranges the Optical Sensor page gives for its own color() method: red 340 to 20, green 80 to 140, blue 200 to 240.
COLOUR_GATE_BRIGHTNESS	10.0	MEASURE: a reading darker than this means nothing is there.
COLOUR_CAL_SAMPLES	20	Colour calibration (RUN_MODE "colour_calibrate"). Readings for the empty claw and for each colour.
COLOUR_CAL_MIN_GOOD	15	Bright enough readings each colour needs.
COLOUR_CAL_READ_MS	50	Gap between calibration readings.
COLOUR_MIN_BRIGHTNESS_GAP	5.0	A ball reading must beat the empty claw by this much.
COLOUR_SPREAD_FACTOR	3.0	Tolerance = this many times the spread of the readings.
COLOUR_MIN_TOLERANCE_DEG	8.0	
COLOUR_MAX_TOLERANCE_DEG	30.0	
COLOUR_FILE	"colours.txt"	The calibration, in the SD card's root folder.
COLOUR_LOG_FILE	"colour_log.csv"	Every calibration reading, for tools/fit_colours.py.

Arm and claw

Positions are motor degrees, counted from where the arm and claw stop when they are homed at the start of a sort: the arm all the way down and the claw wide open. The arm motor drives a 12 tooth gear into the 60 tooth gear on the arm (EXP Clawbot build instructions), so 5 motor degrees lift the arm about 1 degree. The sensor check shows both positions while you move them by hand, which is how to find your values.

Setting	Value	Notes
ARM_SEARCH_DEG	0.0	MEASURE: arm while looking for balls (claw down and open).
ARM_GRAB_DEG	0.0	MEASURE: arm when the claw closes on a ball.
ARM_CARRY_DEG	150.0	MEASURE: ball clear of the floor and of the Distance Sensor.
ARM_DROP_DEG	400.0	MEASURE: bottom of the ball above the box rim.
ARM_KP	0.5	Percent per degree of arm error.
ARM_MAX_PCT	40.0	
ARM_MIN_PCT	8.0	MEASURE: lowest power that still moves the arm.

Setting	Value	Notes
ARM_TOLERANCE_DEG	10.0	
ARM_HOME_PCT	15.0	Gentle push down onto the arm's lowest point.
CLAW_HOME_PCT	15.0	Gentle push open onto the claw's open stop.
CLAW_CLOSE_PCT	25.0	Grip power; the claw stops when it meets the ball.
CLAW_OPEN_PCT	25.0	
CLAW_EMPTY_CLOSED_DEG	120.0	MEASURE: where the claw stops when it closes on nothing.
CLAW_GRIP_MARGIN_DEG	20.0	Stopping this far short of that means a ball is held.
STALL_DEG	3.0	A motor that turns less than this ...
STALL_MS	300	... in this long has reached a stop.

Sorting

RUN_MODE "sort". src/mission.py describes the states and what happens when something goes wrong. The robot searches along SELECTED_PATH.

Setting	Value	Notes
APPROACH_PCT	15.0	Slow drive up to a ball.
APPROACH_STOP_MM	100.0	Stop when the ball is this close to the Distance Sensor.
APPROACH_MAX_MM	700.0	Give up if the ball is not reached within this distance.
OBJECT_LOST_MS	1000	No valid distance for this long means the object has gone.
CREEP_PCT	8.0	Then creep forward until the Optical Sensor sees the ball.
CREEP_MAX_MM	150.0	
IN_CLAW_READS	3	Bright readings in a row that mean the ball is in the claw.
GRAB_TRIES	2	
GRAB_RETRY_BACKUP_MM	100.0	
OBSTACLE_STOP_MM	150.0	Anything this close that is not a ball to fetch stops the robot.
IGNORE_RADIUS_MM	200.0	After giving up on an object, ignore anything this near it.
BOARD_MARGIN_MM	0.0	An object only counts as a ball if it is on the board.
UNKNOWN_COLOUR_ACTION	"reject"	"reject": take it to REJECT_POSITION; "stop": stop there.
BOX_STAGING_MM	400.0	Line up this far back from the drop point, and back out to it.
DROP_REACH_MM	310.0	MEASURE: turning centre to the held ball's centre, arm at ARM_DROP_DEG.
DROP_INSET_MM	90.0	Put the ball's centre this far inside the box's near edge.
RELEASE_WAIT_MS	800	
RELEASE_TRIES	2	
NAV_TRIES	3	Times to ask for the way to be cleared before giving up.
ARRIVE_TOLERANCE_MM	30.0	This near a point counts as being there already.

Brain screen

The assignment program showed five lines of up to 16 characters, so the messages are kept that short.

Setting	Value	Notes
SCREEN_PERIOD_MS	250	How often the status lines are redrawn.
SCREEN_ROWS	5	

References

The official VEX pages this project relies on. They are linked, not copied: there are no VEX PDFs, knowledge base articles, images, diagrams or library code in this repository. Every link below was opened on 7 November 2026.

VEX EXP Python API

These pages describe the `vex` module that VEXcode EXP and the EXP Brain provide. src/hardware.py only makes calls listed on them.

Page	What this project takes from it
Python API home	The list of API pages
MicroPython Libraries	The EXP Brain runs MicroPython 1.13, based on Python 3.4, so the build rejects newer Python syntax
Motor and Motor Group	`Motor(port, GearSetting.RATIO_18_1, reversed)`, `spin`, `stop`, `set_stopping` with `BRAKE` and `HOLD`, `position`, `set_position`, `current`
Inertial	The gyro inside the Brain: `calibrate`, `is_calibrating`, `heading`, `set_heading`
Distance Sensor	`is_object_detected`, `object_distance` in mm, and its range of 20 mm to 2000 mm
Optical Sensor	`hue` from 0 to 359, `brightness` as a percent, `is_near_object`, `set_light_power` in whole percent, and the `color()` hue ranges the default `COLOUR_REFERENCES` come from
Brain buttons	`buttonCheck`, `buttonLeft` and `buttonRight` with `pressing()`
Screen	`clear_screen`, `set_cursor`, `print`
Sound	`play_sound(SoundType.TADA)`
Control	`wait(time, MSEC)`
Timer	`brain.timer.time(MSEC)`, which starts from 0 when a project starts
SD Card	`is_inserted`, `loadfile` and `savefile` with a `bytearray`, and the card rules: FAT32, 32 GB or smaller, files in the root folder
Console	`print()` output appears in the VEXcode console

Building the robot and the software

Page	What this project takes from it
EXP build instructions	The EXP Clawbot build, revision 3: the motor ports (6, 10, 3 and 4), the 3.25 in wheels and the 12 tooth to 60 tooth arm gear
EXP Clawbot build	VEX's own link to the Clawbot instructions, from the page above
Install VEXcode for VEX EXP	Getting VEXcode EXP, which downloads the program to the Brain

Copyright and licences

Page	What it says, in short
VEX copyright notice	Innovation First, Inc. (VEX Robotics) owns all content and intellectual property relating to VEX Robotics and its materials, with all rights reserved
VEX terms of use	Content on VEX's site may not be copied, shared, published or turned into derivative works, except where VEX says so explicitly, and then only for personal, non-commercial use with VEX's attributions. Statutory copyright exceptions still apply
VEXcode software licence	VEXcode, its firmware and related materials stay VEX's property. The licence does not allow redistributing, publishing or reverse engineering them

Where this project touches VEX code

- src/hardware.py, and so `brain/clawbot.py`, imports the `vex` module that VEXcode and the Brain provide. That module is VEX's software under the VEXcode licence above. It is not in this repository.

- tests/fake_vex.py is a stand-in for the tests, written from the API pages above. It repeats the names and arguments of the calls this project uses, and none of VEX's code.

This project is not affiliated with or endorsed by VEX Robotics.